

[49] Billion-Scale Similarity Search with GPUs

요약

본 논문은 Jeff Johnson, Matthijs Douze, Hervé Jégou가 2019년에 IEEE Transactions on Big Data에 발표한 논문으로, Faiss 라이브러리의 GPU 가속 기능을 상세히 다루고 있다. 수십억 개(billion-scale) 규모의 벡터 데이터에 대해 초저지연(ultra-low latency)의 유사성 검색을 수행하는 기술을 소개한다.

논문의 핵심은 GPU의 병렬 처리 능력을 최대한 활용하여, 전통적으로 불가능했던 대규모 검색을 실시간으로 수행하는 것이다. CPU 기반의 순차 처리로는 수십억 개의 벡터를 검색하는 데 수 초 이상이 걸리지만, GPU를 활용하면 수십 밀리초 수준으로 단축할 수 있다. 이는 웹 규모의 이미지 검색, 추천 시스템, 일반 유사성 검색 등 다양한 응용에서 혁신적인 변화를 가져온다.

논문은 단순히 GPU 가속만을 다루는 것이 아니라, billion-scale에서 발생하는 다양한 실무적 문제들을 해결하는 방법을 제시한다. 메모리 관리, 배치 처리, 여러 GPU의 효율적 활용, 네트워크 통신 최적화 등이 포함된다. 특히 메모리 제약을 극복하기 위해 PQ (Product Quantization)와 같은 압축 기법을 GPU 레벨에서 효율적으로 구현하는 방법을 보여준다.

상세분석

49.1 GPU의 병렬 처리 특성과 벡터 검색

현대적인 GPU는 수천 개의 병렬 처리 코어(parallel processing cores)를 갖추고 있다. NVIDIA의 고성능 GPU는 대략 5,000 개 이상의 CUDA 코어를 가지고 있으며, 이들은 독립적으로 동작할 수 있다. 벡터 간의 거리 계산은 본질적으로 병렬화 가능한 작업이므로, GPU의 성능을 최대한 활용할 수 있다.

거리 계산의 병렬화는 다음과 같이 진행된다. 데이터 벡터 집합을 여러 개의 블록으로 분할하고, 각 블록의 벡터들과 쿼리 벡터 간의 거리를 병렬로 계산한다. NVIDIA의 CUDA 프로그래밍 모델에서는 각 블록의 처리를 스레드 블록(thread block)으로 구현하고, 각 벡터의 거리 계산을 개별 스레드(thread)로 할당한다.

GPU 메모리의 계층 구조도 중요하다. GPU는 레지스터(register), 공유 메모리(shared memory), 글로벌 메모리(global memory) 등 여러 단계의 메모리를 가지고 있다. 가장 빠른 성능을 위해서는 접근 패턴(access pattern)을 신중하게 설계하여, 자주 접근하는 데이터를 빠른 메모리에 유지해야 한다.

Coalesced memory access는 GPU 성능의 핵심이다. 여러 스레드가 메모리에서 연속된 주소를 접근할 때 (coalesced access), 한 번의 메모리 트랜잭션으로 여러 데이터를 가져올 수 있다. 따라서 벡터 데이터의 메모리 레이아웃을 신중하게 설계하는 것이 중요하다.

49.2 Billion-Scale에서의 메모리 관리

CPU의 주 메모리(main memory)가 수백 GB에서 TB 단위인 반면, GPU의 메모리는 일반적으로 4GB에서 40GB 정도이다. 따라서 billion-scale의 벡터 데이터를 GPU에 모두 저장할 수 없다. 이 문제를 해결하기 위해 논문은 다음과 같은 전략들을 제시한다.

첫째, 벡터 압축이다. PQ (Product Quantization)를 사용하면 128차원 벡터를 4바이트로 압축할 수 있다. 따라서 1억 개의 벡터는 400MB에 저장되며, 10억 개도 4GB에 저장할 수 있다. 하지만 이는 정확도 손실을 초래한다.

둘째, 계층화된 인덱싱이다. 먼저 대규모 데이터를 여러 개의 샤드(shard)로 분할한다. 각 샤드는 개별적으로 GPU 메모리에 적재되거나 디스크에 보관된다. 검색 시에는 필요한 샤드만 GPU로 로드하여 검색을 수행하고, 결과를 병합한다.

셋째, 스트리밍 처리(streaming)이다. 벡터 데이터를 한 번에 모두 메모리에 로드하지 않고, 배치 단위로 순차적으로 처리한다. 이를 통해 메모리 제약을 극복할 수 있다.

메모리 대역폭(memory bandwidth)도 중요한 병목이다. GPU의 메모리 대역폭은 CPU보다 높지만, 여전히 billion-scale의 데이터를 빠르게 처리하기에는 제한적일 수 있다. 따라서 메모리 접근을 최소화하는 알고리즘 설계가 중요하다.

49.3 배치 처리와 처리량 최적화

GPU의 성능을 최대한 활용하려면, 배치 처리(batching)가 필수적이다. 단일 쿼리를 처리하는 데는 GPU의 모든 코어를 활용하기 어렵지만, 여러 개의 쿼리를 동시에 처리하면 GPU의 병렬 처리 능력을 충분히 활용할 수 있다.

배치 크기(batch size)는 GPU 메모리와 성능의 트레이드오프를 결정한다. 배치 크기가 클수록 처리 효율은 높지만, 메모리 사용량도 증가한다. 또한 배치 크기가 크면 응답 시간(latency)이 증가할 수 있다.

논문에서 제시하는 최적화 전략 중 하나는 메모리 재사용(memory reuse)이다. 쿼리 벡터들이 이미 GPU 메모리에 로드되어 있으면, 데이터 벡터를 순차적으로 로드하면서 여러 번 재사용할 수 있다. 예를 들어, 128개의 쿼리를 GPU에 로드하고, 데이터 벡터를 1000개씩 배치로 로드하면서 모든 쿼리 대 데이터 벡터 간의 거리를 계산할 수 있다.

또 다른 최적화는 비동기 처리(asynchronous processing)이다. CUDA의 스트림(stream) 기능을 사용하면, CPU와 GPU 간의 데이터 전송과 GPU 계산을 겹칠(overlap) 수 있다. 이를 통해 전송 지연 시간을 숨길 수 있다.

49.4 다중 GPU 시스템에서의 확장성

단일 GPU로는 billion-scale의 모든 벡터를 저장하고 처리할 수 없으므로, 여러 GPU를 사용하여 확장하는 방법이 필요하다. 논문은 데이터 병렬화(data parallelism)를 기반으로 하는 확장 전략을 제시한다.

기본 아이디어는 벡터 데이터를 여러 GPU에 분산 저장하는 것이다. 각 GPU는 자신에게 할당된 벡터 부분집합에 대해 검색을 수행한다. 검색 요청이 들어오면, 모든 GPU에 병렬로 쿼리를 보내고, 각 GPU는 독립적으로 검색을 수행한 후 결과를 반환한다. 호스트 시스템은 이 결과들을 병합하여 최종 결과를 생성한다.

이 방식의 성능은 GPU 간의 통신 오버헤드에 크게 영향을 받는다. GPU 간 통신이 느리면 전체 성능이 저하된다. 따라서 효율적인 통신 프로토콜이 중요하다. NVIDIA의 NVLink 기술은 GPU 간 고속 통신을 제공하며, 이를 통해 여러 GPU 간의 데이터 전송을 크게 가속할 수 있다.

49.5 IVF+PQ의 GPU 최적화 구현

IVF+PQ는 billion-scale 검색의 가장 널리 사용되는 방법이다. IVF는 대규모 데이터를 먼저 대략적으로 클러스터링하고, PQ는 각 벡터를 압축한다.

GPU에서 IVF+PQ를 구현할 때의 핵심 과정은 다음과 같다. 먼저, 쿼리 벡터가 주어지면 모든 클러스터 중심과의 거리를 계산한다. 이 계산은 GPU에서 매우 빠르게 수행될 수 있다 (수천 개의 클러스터여도 밀리초 수준).

쿼리와 가장 가까운 클러스터들을 선택한다 (예: 상위 100개). 그 다음, 선택된 클러스터들의 벡터들을 대상으로 정밀 검색을 수행한다. 이때 PQ 압축된 벡터들과의 거리를 계산하는데, 이는 테이블 룩업을 기반으로 매우 효율적으로 수행된다.

최종적으로 상위 k개의 벡터를 선택한다. 경우에 따라, 근사 검색 결과의 정확도를 높이기 위해 상위 k개 벡터의 원본 거리를 재계산하는 과정도 거칠 수 있다.

49.6 실험 결과와 성능 특성

논문의 실험은 매우 대규모 데이터셋에서 수행된다. SIFT 1B (10억 개 SIFT 디스크립터), 1년간의 웹 기반 이미지 벡터 등이 사용된다. GPU를 사용한 결과는 인상적이다.

예를 들어, SIFT 1B 데이터셋에 대해 GPU 기반 IVF+PQ 검색은 1초에 수백 개의 쿼리를 처리할 수 있다. 이는 개별 쿼리당 밀리초 수준의 응답 시간을 의미한다. 동일한 데이터셋을 CPU로 검색하면 초 단위의 응답 시간이 필요하므로, GPU는 100배 이상의 성능 향상을 제공한다.

메모리 효율성도 우수하다. PQ 압축을 사용하면 10억 개의 벡터를 수십 GB의 메모리로 저장할 수 있으며, GPU 메모리만으로도 수천만 개의 벡터를 처리할 수 있다.

49.7 본 논문과의 관계

Exqutor는 유사성 쿼리의 카디널리티 추정을 다룬다. 본 논문이 다루는 GPU 기반 billion-scale 검색은 Exqutor가 다루는 카디널리티 추정 문제를 더욱 시급하게 만든다.

GPU를 사용한 검색이 매우 빨라지면서, 쿼리 최적화의 중요성이 상대적으로 더 커진다. 효율적인 실행 계획을 선택하기 위해서는 정확한 카디널리티 추정이 필수적이다. 특히 billion-scale에서는 몇 프로센트의 성능 차이도 초당 수백 쿼리를 처리하는 시스템에서는 무시할 수 없는 차이가 된다.

또한 IVF+PQ의 근사 검색은 결과 크기의 예측을 어렵게 만든다. IVF의 클러스터 선택에서의 근사성과 PQ의 거리 변형이 누적되어, 최종 결과 크기를 정확하게 예측하기 어렵다. Exqutor의 머신러닝 기반 추정 기법은 이러한 복잡한 시스템에서도 정확한 카디널리티 추정을 제공한다.

추가 제기 문제

1. **메모리 대역폭의 한계:** GPU 메모리 대역폭은 높지만, billion-scale 데이터에서는 여전히 병목이 될 수 있다. 메모리 대역폭을 더욱 효율적으로 활용하는 방법이 필요하다.
2. **정확도와 속도의 트레이드오프:** GPU 가속은 속도를 극적으로 높이지만, 검색 정확도와 트레이드오프가 존재한다. PQ 압축 수준을 높이면 메모리와 속도는 이점을 얻지만 정확도가 낮아진다.
3. **동적 업데이트의 어려움:** billion-scale 데이터셋에서 새로운 벡터를 추가하거나 기존 벡터를 삭제하는 것은 복잡한 작업이다. 인덱스를 효율적으로 유지하면서 동시성을 제어하는 방법이 필요하다.
4. **카디널리티 추정의 정확성:** GPU 기반 근사 검색에서 결과 크기의 정확한 예측은 매우 어렵다. 쿼리 최적화를 위해 학습 기반의 카디널리티 추정이 필수적이다.